

AI Transformation Portal

Architecture & Data Flow — Technical Documentation

المنصة الحكومية لتخطيط ومتابعة مشروع الذكاء الاصطناعي المساعد

Document 1 of 2 · Prepared for the IT team (staging deployment, DB linking, go-live)

Version 2.0 · July 2026 · Companion documents: docs/it-handover.md, .env.example, Roles & Access (PDF)

1. System overview

The portal is a **Next.js 14** application (App Router, TypeScript, RTL Arabic UI) backed by **PostgreSQL 16** through **Prisma ORM**. Users sign in with **UAE PASS** (OIDC); a server-side **sessions** table backs the signed-in session. An optional internal, OpenAI-compatible model powers the AI review step. One codebase serves all **five roles** — مشرف النظام (admin), رئيس المسار, منسق المسار في الجهة, ممثل الجهة, and اللجنة الوطنية; every screen derives from a single role-scoped view-model, so what a user can see is decided in one place. The **admin** gets a dedicated console for managing users & roles and assigning the stream heads and national committee.

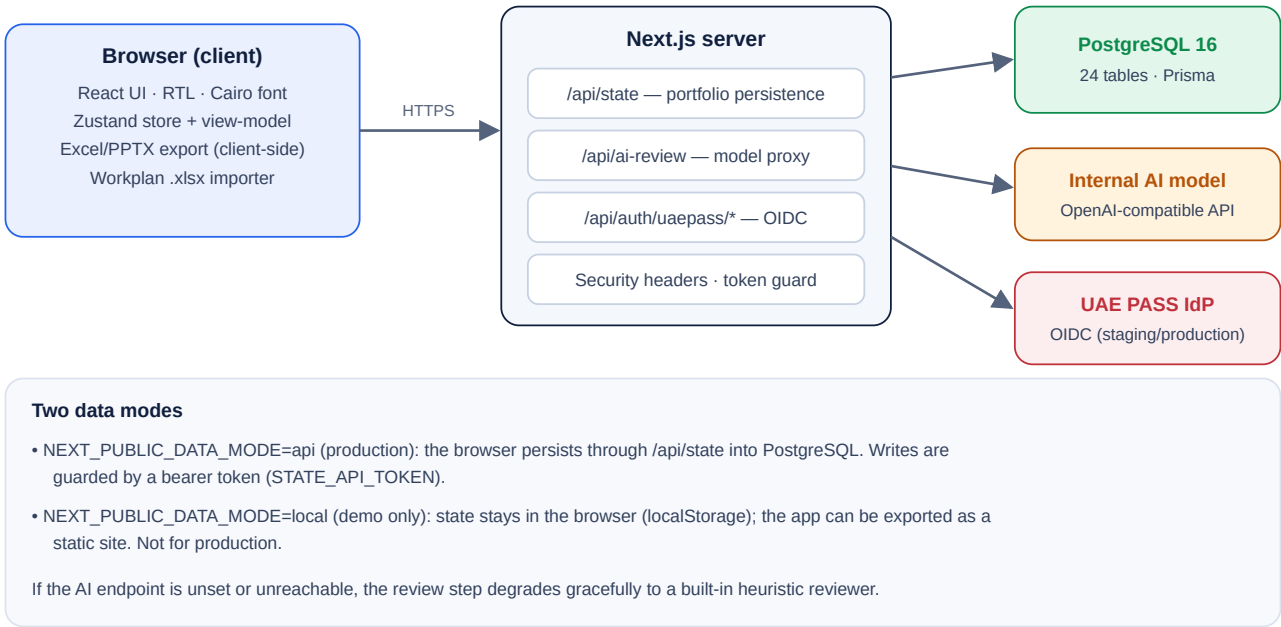


Fig. 1 — Runtime architecture

2. Deployment architecture (Kubernetes)

Manifests are provided in **k8s/**. The application image is built from the repository **Dockerfile**; database schema changes run as a one-shot migration **Job** before each rollout.

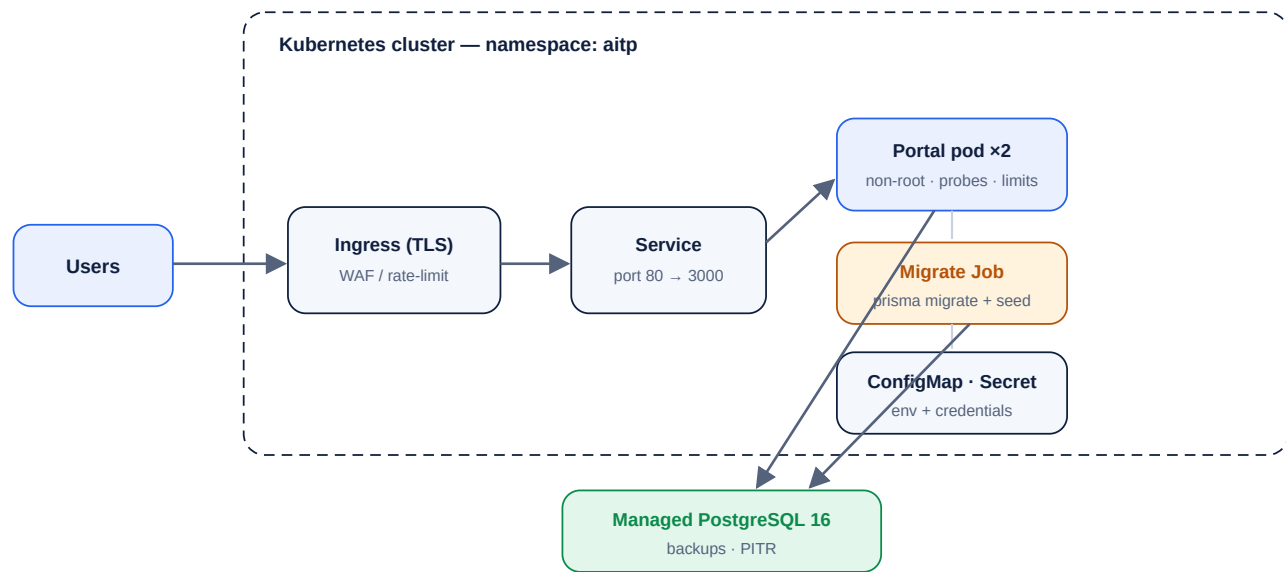


Fig. 2 — Kubernetes deployment (k8s/ manifests)

Release flow

#	Step	Command / artifact
1	Build & push image	<code>docker build -t REGISTRY/aitp-portal:TAG .</code>
2	Apply namespace, config, secret	<code>k8s/namespace.yaml · configmap.yaml · secret (from secret.example.yaml)</code>
3	Migrate + seed reference data	<code>k8s/migrate-job.yaml (idempotent)</code>
4	Roll application	<code>k8s/deployment.yaml · service.yaml · ingress.yaml</code>

3. Data model

Schema source of truth: **prisma/schema.prisma** + migrations **0001–0008** (verified on a fresh PostgreSQL 16 — 24 tables). The seed loads **reference data + roles + starter accounts**; items, launch plans and all transaction tables start empty (the demo portfolio only seeds when NEXT_PUBLIC_DEMO_DATA=1).

Group	Tables	Notes
Reference	streams, entities, program_phases, exec_batches, settings	5 streams in official order with the predefined heads (رؤساء المسارات); 35 federal entities; المراحل; approved committee budget (100M AED).
Access control	roles, users, sessions	roles holds the 5 roles (admin coord entity path ai) with a permissions JSON; users.role is a FK into it. Seed creates starter accounts (admin, committee, 5 stream heads, entity rep, 5 coordinators) keyed by @aigp.gov.ae emails IT re-points to UAE PASS identities. sessions backs each signed-in session.
Team setup	entity_reps, stream_owners	Per-entity representative and per-stream owners; the entity rep provisions its own coordinators here.
Portfolio	items, item_launch_plans, launch_plans, launches, item_launches, exec_checklist_items, sub_milestones	Items carry the execution budget ; a launch plan's execution budget is derived (sum of attached items) and its launch_budget is informational only.
Workflow / audit	log_entries, nominations, fundings, funding_cancellations, notifications	Every action is logged with actor + timestamp; nominations and funding decisions are first-class rows. notifications persist الإشعارات (per-user or role/entity/stream broadcast) for optional email/SMS push.
Misc	app_state	Serialized UI-state snapshot used by /api/state.

Cost model (important)

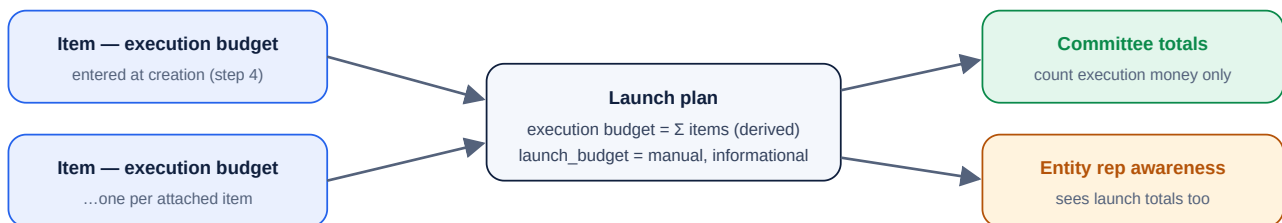


Fig. 3 — Cost derivation: items feed plans; funding counts execution money only

4. Data flow — sign-in (UAE PASS OIDC)

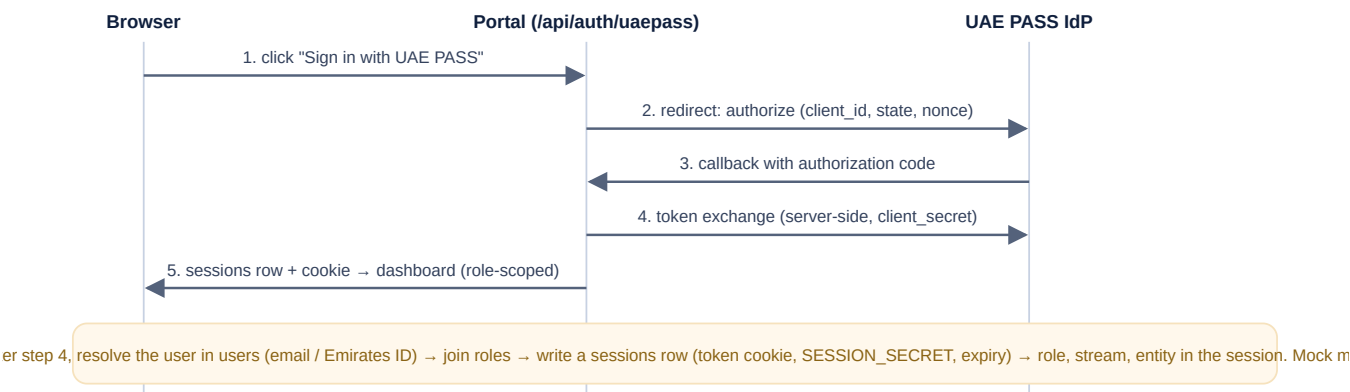


Fig. 4 — Authentication sequence

5. Data flow — item lifecycle

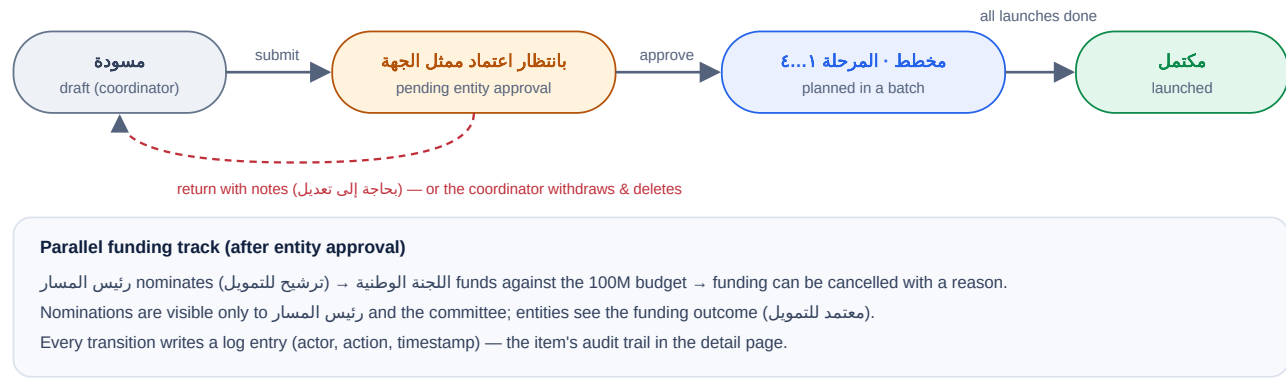


Fig. 5 — Lifecycle state machine + funding track

6. Data flow — bulk import (workplan file)

The coordinator uploads the **official خطة العمل Excel export** that entities already produce (the same file from the existing workplan system). The importer reads every sheet and maps the legacy vocabulary automatically.

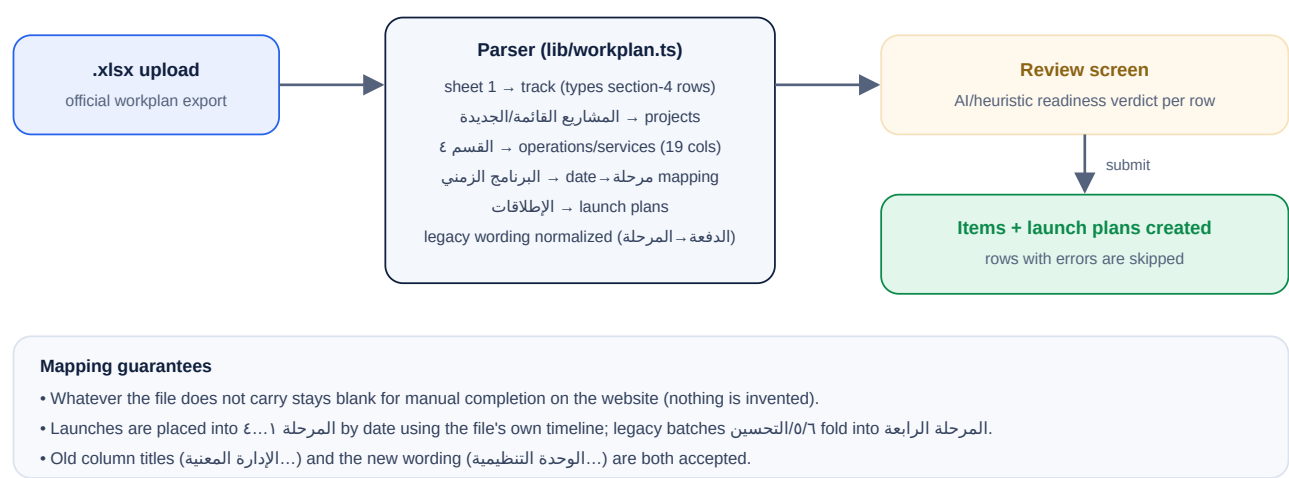


Fig. 6 — Bulk import pipeline

7. Data flow — persistence & exports

Flow	Path	Notes
State write	UI action → store → POST /api/state → PostgreSQL	Bearer-token guarded; every workflow action also appends a log entry.
State read	App load → GET /api/state → store hydrate	Role scoping is applied in the view-model on every render.
AI review	UI → POST /api/ai-review → internal model	Server-side proxy; heuristic fallback if the model is unavailable.
Excel / PowerPoint export	Client-side generation (exceljs / pptxgenjs)	Exports the current role-scoped list only — a user can never export data they cannot see.

8. Access provisioning & the admin console

Accounts live in **users** (FK **users.role** → **roles.key**). Two actors provision people, and the split is enforced by role: the **admin** owns platform-level identities (stream heads and the national committee), while each **entity rep** owns the coordinators inside its own entity.

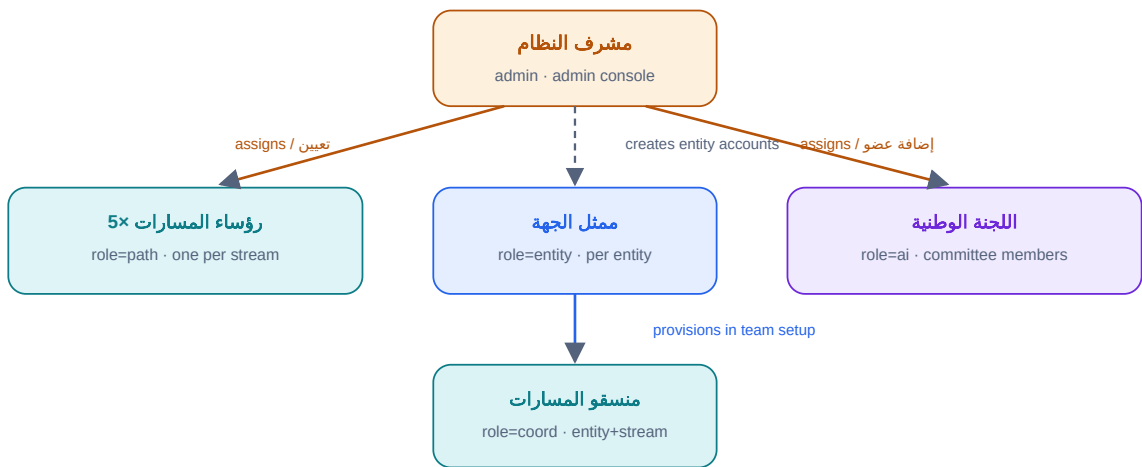


Fig. 7 — Who provisions whom (admin console vs. entity team setup)

Roles & capabilities (roles.permissions)

role	الدور	scope	Key capabilities
admin	مشرف النظام	system	users.manage · roles.view · streamhead.assign · committee.assign
ai	اللجنة الوطنية	national	item.view.all · nomination.review · funding.approve · funding.cancel · phase.edit · budget.set
path	رئيس المسار	stream	item.view.stream · item.nominate · plan.view
entity	ممثل الجهة	entity	item.view.entity · item.approve · item.return · team.manage
coord	منسق المسار في الجهة	entity+stream	item.create · item.update · item.submit · plan.edit

Provisioning rules enforced by the app: **coord** and **entity** require `entity_id`; **coord** and **path** require `stream_id`; **ai** and **admin** require neither. Deactivate (never delete) accounts via `is_active=false` so the audit trail keeps valid author names.